# Enterprise WAN Architect: diseño, automatización y ciberdefensa de redes WAN

**Gerardo Esteban Rojas Manquillo**
Fundación Universitaria Compensar — Interconexión de Redes WAN
gerojas@ucompensar.edu.co
https://github.com/Gera871/redes-wan-rojas

Resumen—Este artículo presenta la evolución de Enterprise WAN Architect durante el tercer corte académico. La herramienta, desarrollada en Python y Streamlit, permite definir escenarios corporativos, calcular subredes, visualizar topologías jerárquicas y generar comandos para Cisco, Huawei, Fortinet y MikroTik. Se describen los mecanismos de ciberdefensa declarados, las evidencias disponibles y las limitaciones detectadas en el reporte de pruebas.

*Palabras clave—WAN, subnetting, Streamlit, IPv4, IPv6, Graphviz, SSH, ciberdefensa.*

## I. INTRODUCCIÓN

En el segundo corte se desarrolló la base de Enterprise WAN Architect: parametrización del escenario, planificación IP, representación topológica y generación de configuraciones. En el tercer corte se documenta el soporte para cuatro fabricantes, las políticas de ciberdefensa y las pruebas exigidas en la guía de la asignatura. El objetivo es reducir tareas manuales y facilitar la revisión de diseños de red antes de su implementación.

## II. OBJETIVOS

### A. Objetivo general

Desarrollar y documentar una herramienta web para el diseño y gestión de redes WAN corporativas, con generación de configuraciones multimarca, controles de seguridad y evidencias de evaluación.

### B. Objetivos específicos

Calcular subnetting IPv4 e incluir bloques IPv6; construir topologías jerárquicas; generar scripts para Cisco IOS, Huawei VRP, Fortinet FortiOS y MikroTik RouterOS; registrar políticas de ciberdefensa; presentar resultados y limitaciones de las pruebas.

## III. ARQUITECTURA Y METODOLOGÍA

La aplicación está construida en Python con Streamlit 1.38.0. Usa ipaddress para los cálculos IPv4, Graphviz DOT para dibujar la topología, Paramiko 3.4.0 para conexiones SSH y archivos JSON para guardar proyectos. El archivo app.py coordina los módulos y assets/topology.py construye el grafo. La configuración del escenario se comparte mediante st.session_state.

El escenario de referencia corresponde a un edificio corporativo de 15 pisos y 250 clientes, con red base 192.168.0.0/22. La aplicación permite cambiar estos parámetros. La topología incorpora ISP, router de borde, router core, switch core, switches por piso y DMZ.

## IV. DIRECCIONAMIENTO Y TOPOLOGÍA

Para una red 10.20.0.0/21, el espacio IPv4 contiene 2048 direcciones. Al dividirla en bloques /26 se obtienen 32 subredes de 64 direcciones, con 62 hosts utilizables por subred; la primera dirección utilizable se reserva como gateway. La interfaz presenta red, máscara, broadcast y hosts. El código también asigna identificadores IPv6 de documentación 2001:db8::/32 por piso; esto no equivale a un cálculo completo de capacidad IPv6.

La representación de red se genera dinámicamente con DOT. Los enlaces WAN se muestran como /30, y la estructura por piso incluye VLAN y gateway. Antes de usar los bloques en producción debe verificarse que no se solapen los rangos LAN y WAN y que exista espacio suficiente para todas las zonas.

## V. CONFIGURACIONES MULTIMARCA

El módulo 4 permite seleccionar un fabricante, definir hostname y VLAN, generar comandos CLI y descargarlos como archivo .txt. Las plantillas usan sintaxis diferenciada: Cisco IOS con vlan e interface Vlan; Huawei VRP con vlan batch e interface Vlanif; Fortinet FortiOS con config system interface; y MikroTik RouterOS con /interface vlan y /ip address.

Las salidas constituyen propuestas de configuración que deben revisarse y adaptarse al modelo, versión de sistema operativo, interfaces y direccionamiento real antes de aplicarse. La aplicación usa en varias plantillas direcciones 192.168.<VLAN>.1/24, no necesariamente derivadas del plan de subnetting.

### A. Fortinet y MikroTik — novedades del corte 3

En Fortinet se genera un bloque config system interface con VLAN, IP y acceso administrativo. En MikroTik se construyen comandos de creación de VLAN, dirección IP y habilitación de SSH. Las capturas originales aportadas para este informe muestran ambas salidas y su descarga como archivos .txt.

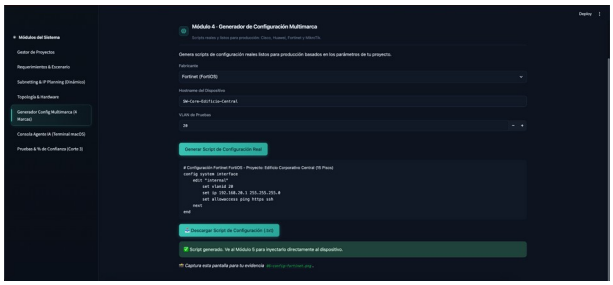



*Fig. 1. Configuración Fortinet FortiOS generada para VLAN 20.*

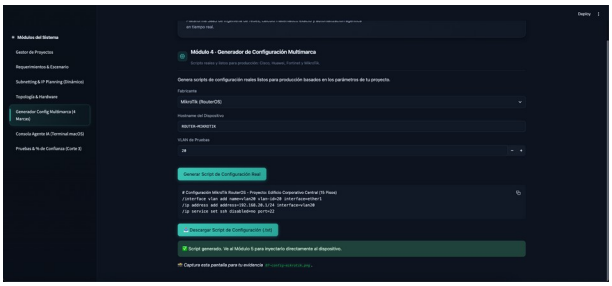


*Fig. 2. Configuración MikroTik RouterOS generada para VLAN 20.*

## VI. CIBERDEFENSA Y DESPLIEGUE

Las políticas del repositorio proponen no publicar secretos, utilizar datos de ejemplo al consultar IA, validar entradas, preferir SSH sobre Telnet y limitar la ejecución a laboratorios autorizados. El código valida direcciones IP en el cálculo de subredes y en la conexión SSH. La captura 08 documenta un error controlado al intentar abrir un puerto serial inexistente; no constituye evidencia de una prueba de rechazo de entradas maliciosas. También admite autenticación por llave privada y conexión serial mediante pyserial.

Se identifican aspectos por fortalecer: la política SSH utiliza AutoAddPolicy para aceptar claves de host desconocidas; la validación de hostname, VLAN y comandos no es integral; no hay evidencia de un control de acceso a la propia aplicación; y el envío de comandos no verifica sistemáticamente su aplicación exitosa. Por tanto, no se considera demostrada una ciberdefensa completa.

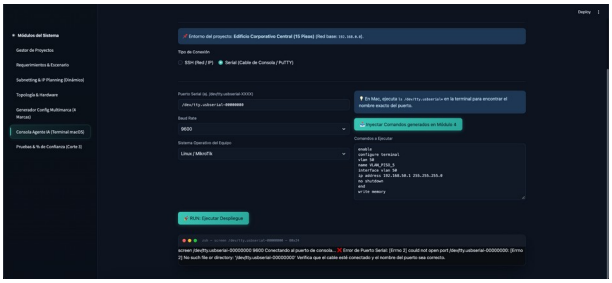


*Fig. 3. Error controlado de conexión a puerto serial inexistente.*

## VII. PRUEBAS Y PORCENTAJE DE CONFIANZA

La guía define el porcentaje de confianza como pruebas superadas divididas entre pruebas totales, multiplicado por 100. El archivo docs/pruebas.md declara 15 de 15 pruebas aprobadas, equivalente al 100 %. Sin embargo, el panel de la aplicación presenta ocho pruebas con estado fijo «Superada» y una métrica fija de 98,5 %. Estas cifras no representan una misma ejecución verificable.

Para sustentar el porcentaje, se requiere ejecutar casos reproducibles, registrar entradas, resultados esperados y obtenidos, y calcular el indicador a partir de los resultados reales. Las pruebas de SSH y consola serial necesitan dispositivos o simuladores autorizados; la existencia del código no demuestra por sí sola conectividad efectiva.

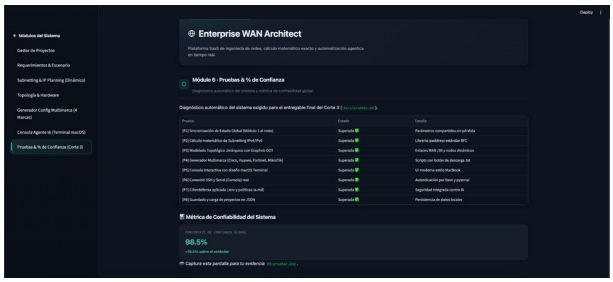


*Fig. 4. Panel de pruebas y métrica mostrada de 98,5 %.*

## VIII. EVIDENCIAS DEL TERCER CORTE

El repositorio contiene las cinco capturas solicitadas: 06-config-fortinet.png, 07-config-mikrotik.png, 08-ciberdefensa.png, 09-pruebas.png y 10-app-final.png. Estas evidencias pueden consultarse en https://github.com/Gera871/redes-wan-rojas/tree/main/docs/capturas. Se incluyen como figuras 1 a 5 dentro de las secciones técnicas correspondientes, sin páginas independientes.

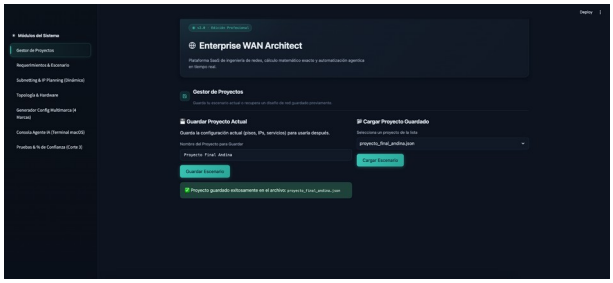


*Fig. 5. Gestor de proyectos y guardado de escenario JSON.*

## IX. CONCLUSIONES

Enterprise WAN Architect integra en una misma interfaz el cálculo de direccionamiento, la representación de topologías y la generación de comandos para cuatro fabricantes. La separación en módulos y el almacenamiento de proyectos facilitan su uso académico. El tercer corte amplía el alcance multimarca y formaliza políticas de seguridad. Para respaldar una sustentación rigurosa se recomienda corregir el indicador de pruebas, automatizar verificaciones y reforzar las validaciones y la seguridad de SSH.

## REFERENCIAS


[1] G. E. Rojas Manquillo, “Enterprise WAN Architect: informe del corte 2,” UCompensar, 2026.

[2] G. E. Rojas Manquillo, “redes-wan-rojas,” GitHub, 2026. https://github.com/Gera871/redes-wan-rojas

[3] Python Software Foundation, “ipaddress — IPv4/IPv6 manipulation library,” Python Documentation.

[4] Streamlit, “Streamlit documentation.” https://docs.streamlit.io

[5] Paramiko, “Paramiko documentation.” https://www.paramiko.org